



ЛОГИЧЕСКАЯ ПАРАДИГМА ЯЗЫКОВ ПРОГРАММИРОВАНИЯ

ДИСЦИПЛИНА ЯЗЫКИ И СИСТЕМЫ ПРОГРАММИРОВАНИЯ
(ПАРАДИГМЫ ЯЗЫКОВ ПРОГРАММИРОВАНИЯ)



Литовка Наталья Васильевна

ПРЕДИКАТЫ, ФАКТЫ И ПРАВИЛА

Предикат — это утверждение, высказанное о субъекте. Субъектом высказывания называется то, о чём делается утверждение.

Предикат в программировании — выражение, использующее одну или более величину с результатом булева типа. Конструкция на языке программирования, предназначенная для выполнения вычислений, комбинация значений, констант, переменных, операций и функций, которая может быть интерпретирована в соответствии с правилами конкретного языка.

Факт — это предложение, у которого нет тела. В терминах логики предикатов, **факт** — это и есть предикат. Он фиксирует (определяет) некоторое отношение между объектами. Например, факт, что Наташа является мамой Даши, может быть записан в виде (в SWI-Prolog строки-константы записываются в одинарных кавычках):

```
мама('Наташа', 'Даша').
```

Факт представляет собой безусловно истинное утверждение. Если воспользоваться нормальной формой Бэкуса-Науэра, то предикат можно определить следующим образом:

$$\langle \text{Предикат} \rangle ::= \langle \text{Имя} \rangle \mid \langle \text{Имя} \rangle (\langle \text{аргумент} \rangle \mid \langle \text{аргумент} \rangle^*),$$

т.е. предикат состоит либо только из имени, либо из имени и следующей за ним последовательности аргументов (термов), заключенной в скобки.

Аргументом (термом) предиката может быть константа, переменная или составной объект (список или функция). Число аргументов предиката называется **арностью**.

Правило — предложение, истинность заголовка которого в виде предиката зависит от истинности одной или нескольких формул, указанных в теле. Обычно правило содержит несколько хвостовых целей, которые должны быть истинными для того, чтобы само правило было истинным. В нотации БНФ правило будет иметь вид:

$\langle \text{Правило} \rangle ::= \langle \text{предикат} \rangle :- \langle \text{предикат} \rangle [, \langle \text{предикат} \rangle]^* .$

Например. Известно, что бабушка человека - это мама его мамы или мама его папы. Соответствующие правила будут иметь вид:

```
grandmama (X, Y) :-  
    mama (X, Z) , mama (Z, Y) .  
grandmama (X, Y) :-  
    mama (X, Z) , papa (Z, Y) .
```

или

```
grandmama (X, Y) :-  
    mama (X, Z) , mama (Z, Y) ;  
    mama (X, Z) , papa (Z, Y) .
```

или

```
grandmama (X, Y) :-  
    mama (X, Z) , (mama (Z, Y) ; papa (Z, Y)) .
```

СВОБОДНЫЕ ПЕРЕМЕННЫЕ И УНИФИКАЦИЯ

Свободная переменная – переменная, которая еще не получила значения. Она не равняется ни нулю, ни пробелу; у нее вообще нет никакого значения. Такие переменные еще называют **неконкретизированными**.

Переменная, которая получила какое-то значение, называется **связанной**. Такой переменной не может быть присвоено новое значение, т. е., по сути, переменная становится константой.

Унификация

Общий принцип выполнения программ на Прологе прост: производится поиск ответа на вопросы, задаваемые БД, состоящей из фактов и правил, то есть проверяется соответствие предикатов вопроса предложениям из БД. Это частный случай метода резолюций.

Отметим, что в Прологе не проводится синтаксического различия между предикатом и функцией (составным термом), а также между нечисловой константой и функцией без аргументов. Следовательно, суть действия состоит в том, что ищется попарное соответствие между термами, один из которых является целью, а другой принадлежит БД.

Установление соответствия между термами является основной операцией при вычислении цели. Она осуществляется следующим образом: на каждом шаге выбирается очередной терм и отыскивается соответствующее выражение в БД. При этом переменные получают или теряют значения. Этот процесс можно описать в терминах текстуальных подстановок: «подставить терм t вместо переменной Y ». Свободными переменными в Прологе называются переменные, которым не были присвоены значения, а все остальные переменные называются связанными переменными. Переменная становится связанной только во время унификации, переменная вновь становится свободной, когда унификация оказывается неуспешной или цель оказывается успешно вычисленной. В Прологе присваивание значений переменным выполняется внутренними подпрограммами унификации. Переменные становятся свободными, как только для внутренних подпрограмм унификации отпадает необходимость связывать некоторое значение с переменной для выполнения доказательства подцели.

Правила унификации:

1. Если x и y -константы, то они унифицируемы, только если они равны.
2. Если x - константа или функция, а Y -переменная, то они унифицируемы, при этом Y принимает значение x .
3. Если x и y -функции, то они унифицируемы тогда и только тогда, когда у них одинаковые имена функций (функторы) и набор аргументов и каждая пара аргументов функций унифицируемы.

Есть особый предикат « $=$ », который используется в Прологе для отождествления двух термов. Использование оператора « $=$ » поможет лучше понять процесс означивания переменной. В Прологе оператор « $=$ » интерпретируется как оператор присваивания или как оператор проверки на равенство в зависимости от того, являются ли значения термов свободными или связанными.

Пример: $X=Y$, если X и Y – связанные переменные, то производится проверка на равенство, например: если $X=5$ и $Y=5$, то результат ДА (истина); если $X=6$ а $Y=5$, то результат НЕТ(ложь). Если одна из переменных X или Y – свободная, то ей будет присвоено значение другой переменной, для Пролога несущественно слева или справа от знака « $=$ » стоит связанная переменная.

Оператор « $=$ » ведет себя точно так, как внутренние подпрограммы унификации при сопоставлении целей или подцелей с фактами и правилами программы.

ПОИСК С ВОЗВРАТАМИ

Поиск с возвратом, бэктрекинг — общий метод нахождения решений задачи, в которой требуется полный перебор всех возможных вариантов в некотором множестве M . Как правило позволяет решать задачи, в которых ставятся вопросы типа: «Перечислите все возможные варианты ...», «Сколько существует способов ...», «Есть ли способ ...», «Существует ли объект...» и т. п.

Незначительные модификации метода поиска с возвратом, связанные с представлением данных или особенностями реализации, имеют и иные названия: метод ветвей и границ, поиск в глубину, метод проб и ошибок и т. д. Поиск с возвратом практически одновременно и независимо был изобретен многими исследователями ещё до его формального описания.

Описание метода:

Решение задачи методом поиска с возвратом сводится к последовательному расширению частичного решения. Если на очередном шаге такое расширение провести не удастся, то возвращаются к более короткому частичному решению и продолжают поиск дальше. Данный алгоритм позволяет найти все решения поставленной задачи, если они существуют. Для ускорения метода стараются вычисления организовать таким образом, чтобы как можно раньше выявлять заведомо неподходящие варианты. Зачастую это позволяет значительно уменьшить время нахождения решения.

Использование метода:

Классическим примером использования алгоритма поиска с возвратом является задача о восьми ферзях. Её формулировка такова: «Расставить на стандартной 64-клеточной шахматной доске 8 ферзей так, чтобы ни один из них не находился под боем другого». Сперва на доску ставят одного ферзя, а потом пытаются поставить каждого следующего ферзя так, чтобы его не били уже установленные ферзи. Если на очередном шаге такую установку сделать нельзя — возвращаются на шаг назад и пытаются поставить ранее установленного ферзя на другое место.

Помимо этого, метод поиска с возвратом позволяет решать множество других переборных задач.

Например, с помощью него можно получить

все подмножества, размещения, перестановки, сочетания данного множества M .

Недостатки:

Метод поиска с возвратом является универсальным. Достаточно легко проектировать и программировать алгоритмы решения задач с использованием этого метода. Однако время нахождения решения может быть очень велико даже при небольших размерностях задачи (количестве исходных данных), причём настолько велико (может составлять годы или даже века), что о практическом применении не может быть и речи. Поэтому при проектировании таких алгоритмов, обязательно нужно теоретически оценивать время их работы на конкретных данных. Существуют также задачи выбора, для решения которых можно построить уникальные, «быстрые» алгоритмы, позволяющие быстро получить решение даже при больших размерностях задачи. Метод поиска с возвратом в таких задачах применять неэффективно.

СООТВЕТСТВИЕ КАРРИ-ХОВАРДА

Соответствие Карри — Ховарда (*изоморфизм Карри — Ховарда*) — наблюдаемая структурная эквивалентность между математическими доказательствами и программами, которая может быть формализована в виде изоморфизма между логическими системами и типизированными исчислениями.

Хаскелл Карри и Уильям Ховард заметили, что построение конструктивного доказательства похоже на описание вычислений, а высказывания конструктивной логики по своей структуре схожи с типами вычисляемых выражений — программ для вычислительной машины. Ранние проявления этой связи — интерпретация Брауэра — Гейтинга — Колмогорова (1925), задающая семантику интуиционистской логики через вычисления доказательств, и теория реализуемости Клини (1945).

Соответствие Карри — Ховарда в современном представлении не ограничивается какой-то одной логикой или системой типов. Например, логика высказываний соответствует простому типизированному λ -исчислению, логика высказываний второго порядка — полиморфному λ -исчислению, исчисление предикатов — λ -исчислению с зависимыми типами.

ССЫЛКИ НА МАТЕРИАЛ

[Предикаты](#)

[Основные понятия Пролога](#)

[Унификация](#)

[Алгоритм перебора с возвратом](#)

[Соответствие Карри-Ховарда](#)